Gap Analysis — Plain English Edition

Date: 2026-04-23

This is the same content as `GAP_ANALYSIS.md`, but every technical term is **kept** (so you can still match it back to the code and to the original brief) and then **explained in plain English right next to it** so you know what it actually means.

> **How to read the explanations:** anything in `code font` is a real name of a file, setting, function, library, or technology. After each one, you'll see a short "= what that actually is" in plain words.

There's also a quick glossary at the very bottom (Section 7) for terms that come up over and over.

How the status icons work

- ✓ **Done** — built, tested, working. You don't need to do anything.
- 🟡 **Stubbed** — the file exists but the code inside is a placeholder (a "stub"). The chatbot will still run, that feature just won't do anything useful until someone finishes it. **Optional polish.**
- ✗ **Not started** — we never built it. Usually on purpose (Section 4).

Only **Section 3** requires you to do something before turn-on.

---

1. Foundations (A-series) — the database layer

> "A-series" just means "the first batch of things we built." A1, A2, etc. are our internal task IDs.

ID	Asked for	Status	Where it lives + what it means
A1	**Postgres 16 schema** — 27 tables across 8 groups	✓	scripts/setup_database.sql. *Postgres = the brand of database we're using. A "schema" = the blueprint of which tables exist and what columns they have. 27 tables = 27 different "spreadsheets" inside the database.*
A2	**pgvector extension + ivfflat index** on `document_chunks.embedding`	✓	setup_database.sql §"Extensions" + §"Indexes". *pgvector = an add-on to Postgres that lets it store and search "embeddings" (lists of numbers that represent the meaning of text). ivfflat = a fast lookup method for those embeddings.* `document_chunks` *= the table that holds pieces of documents.* `embedding` *= the column holding those number lists.*

A3	**Reference data** — `failure_modes`, `equipment_taxonomy`, `prompt_versions`	✓	scripts/seed_reference_data.sql. *"Reference data" = the lookup lists the chatbot consults.* `failure_modes` *= the list of defect names like "delamination".* `equipment_taxonomy` *= the list of equipment names.* `prompt_versions` *= different wordings of the instructions we give the AI.*
A4	Initial **line memory** + canned **process facts**	✓	service/scripts/seed_initial_data.py. *"Line memory" = short notes about your specific production line ("Coater 1 has 5 zones"). "Process facts" = engineering knowledge about how the process normally behaves.*
A5	**BM25 sparse index** (GIN on `to_tsvector('english', chunk_text)`)	✓	setup_database.sql — `idx_chunks_bm25_gin`. *BM25 = a classic keyword-search algorithm (think Ctrl-F on steroids). "Sparse" = it only looks at the words actually in the text. "GIN" = the type of Postgres index that makes keyword search fast.*
A6	**Materialized view** `v_rca_precision_daily` for outcome closure tracking	✓	setup_database.sql — refresh wired into nightly loop in service/main.py. *"Materialized view" = a pre-computed summary table that gets refreshed on a schedule. Here it's a daily report card of "how often was the chatbot's root-cause guess actually right?"*
A7	Audit + feedback tables (`message_feedback`, `user_corrections`, `outcome_linkages`, `audit_log`)	✓	setup_database.sql §"Audit & Feedback". *Storage for thumbs-up/thumbs-down, operator corrections, what actually fixed each problem, and a log of every action for compliance.*

---

2. Service capabilities (B-series) — the chatbot's brain

"Service" = the running program that takes a question and produces an answer. "B-series" = our second batch of task IDs.

ID	Asked for	Status	Where it lives / what it means
B0	**Deterministic tool layer** (`percentile_of`, `compare_to_distribution`, `nearest_historical_runs`, `detect_drift`, `defect_events_in_window`, `chunk_search`)	✓	service/services/tools.py. *"Deterministic" = same input always gives the same output (no AI guessing). These are six little math/lookup helpers the AI can call. E.g.* `percentile_of` *= "where does this number rank vs. history?",* `chunk_search` *= "find me documents about X."*

B0.5	**Tool-calling LLM loop** with citation collection + token accounting	✓	service/services/llm.py `_run_tool_loop`. *"LLM" = Large Language Model = the AI brain (e.g. GPT-4). "Tool-calling loop" = the AI is allowed to pause mid-answer, call one of the B0 tools, get a real number back, then keep writing. "Citations" = the AI must list which documents/numbers it used. "Token accounting" = tracking how much the OpenAI bill is.*
B1	**Hybrid retrieval** — vector + BM25 fused via **RRF**, FM/equipment filter, **MMR** diversification	✓	service/services/retrieval.py `retrieve_chunks_hybrid`. *"Retrieval" = finding relevant documents. "Hybrid" = combining two search methods. "Vector" = meaning-based search (uses the embeddings from A2). "BM25" = keyword search (from A5). "RRF" (Reciprocal Rank Fusion) = the math formula that merges the two ranked lists. "FM/equipment filter" = "only look at documents about this failure mode and this equipment." "MMR" (Maximal Marginal Relevance) = removes near-duplicate results so you don't get 5 copies of the same paragraph.*
B2	**Cross-encoder reranker** over top-N hybrid candidates	◍	service/services/reranker.py — pass-through stub. *"Reranker" = a second, smarter AI that re-orders the top search results. "Cross-encoder" = the type of model that reads the question and the document together for a more accurate score.* ***Needed:*** *install the `sentence-transformers` Python library, load the `BAAI/bge-reranker-base` model, score `(query, chunk_text)` pairs, reorder. Stub file has full instructions inline.*
B3	**Structure-aware chunker** (preserves headings, tables, bullet lists)	✓	service/services/chunker.py. *"Chunker" = the thing that breaks long documents into bite-size pieces before storing them. "Structure-aware" = it doesn't cut paragraphs or tables in half.*
B4	**Query rewriter** (multi-query + step-back abstraction)	◍	covered by B0.5 tool loop; standalone rewriter not separated. *"Query rewriter" = automatically rewording the operator's question into 2–3 variations before searching. "Step-back" = also asking a more general version of the question. Currently the AI does this on its own when it needs to.*

B5	**HyDE** (Hypothetical-Document Embedding) for cold-start queries	◍	not implemented. *HyDE = "make the AI write a fake answer first, then search for documents that look like that fake answer." Helps when the question doesn't share words with any document. "Cold-start" = the document library is brand new / nearly empty.*
B6	**Self-consistency / k-sample voting** for high-stakes answers	◍	not implemented. *Run the same question through the AI 3–5 times (k samples), take the majority answer. Costs k× more money but is more reliable.*
B7	**Multivariate anomaly detection** over the curated tag block	✓	service/services/deviation.py (renamed from `anomaly.py`). *"Multivariate" = looking at many sensors together at once. "Anomaly detection" = spotting when the combination of readings is weird, even if no single reading is. "Curated tag block" = the hand-picked list of important sensor readings.*
B8	**Two-step RCA reasoning chain** (hypothesize → adjudicate) + 2 prompts	✓	service/services/rca.py + `config/prompts/rca_step{1,2}_v1.txt`. *"RCA" = Root Cause Analysis. Step 1: AI lists 3 candidate causes ("hypothesize"). Step 2: AI weighs them and picks the best one ("adjudicate"). Each step has its own prompt = a separate written instruction.*
B9	**Change ledger** — what changed since baseline (recipe, crew, shift, equipment)	✓	service/services/change_ledger.py; crew/shift now wired through `RecipeContext`. *"Baseline" = "the last time things were running fine." "Change ledger" = a list of every difference between then and now. `RecipeContext` = the bundle of data describing what's currently being made.*
B10	**Outcome closure** — 24h follow-up sweep + precision view refresh	✓	service/services/outcomes.py + nightly hook in main.py. *Every night, a job goes back through the chatbot's recent advice and asks "did this actually fix the problem?" then refreshes the report card from A6.*
B11	**Active-learning loop** (correction → embedding boost / chunk demotion)	◍	feedback API stores signals; the loop that consumes them and adjusts ranking lives in retrieval.py (bounded ±30%); the explicit "active learning trainer" job is not implemented. *"Active learning" = the chatbot getting smarter from operator feedback. "Embedding boost" = pushing helpful documents up in search. "Chunk demotion" = pushing unhelpful documents down. "Bounded ±30%" = the adjustment can't move a document more than 30% up or down (safety limit).*

B12	**Local vLLM provider** as a swap-in for OpenAI	✓	service/services/llm.py `LocalVLLMClient`, registered in `provider_for(...)`. *"vLLM" = an open-source server that runs AI models on your own hardware. "Provider" = the chatbot can be told to use OpenAI **or** vLLM by changing one setting — useful if you don't want OpenAI bills or can't send data to the cloud.*
B13	**Evaluation harness** — replay golden cases, score citation P/R, FM accuracy	🔘	service/eval/harness.py — three `NotImplementedError` stubs with full implementation notes. *"Evaluation harness" = an automated quiz for the chatbot. "Golden cases" = a set of practice questions with known-correct answers. "Citation P/R" = Precision/Recall on which sources it cited (did it cite the right docs? did it miss any?). "FM accuracy" = did it pick the right failure mode? `NotImplementedError` = Python's way of saying "this function is empty on purpose, fill me in."*
—	**Symphony video capture adapter**	🔘	service/services/symphony_capture.py returns `extraction_status: "stub"`. *Symphony = the video system on your line. "Adapter" = a connector that pulls video transcripts in. **Needs your Symphony API endpoint + auth credentials** to do anything real.*

---

3. Placeholders that MUST be replaced before go-live

> "Placeholder" = a fake value we put in so the file would still load during development. You **must** swap each one for a real value before the chatbot is useful.

Each entry tells you: **(a)** the file + line, **(b)** the current placeholder value, **(c)** what to replace it with, **(d)** where to find that value.

3.1 Service config — `service/config/settings.py`

> "Service config" = the main settings file for the chatbot brain. Open it in any text editor (Notepad, VS Code, etc.).

Setting	Current (placeholder)	Replace with	Source (where to

`api_key`	`"dev-key-change-me"` *(literally a note saying "change me")*	A 32+ character random string. *This is the password Ignition will use to talk to the chatbot — like a shared secret handshake.*	Run `python -c "ir` `print(secrets.to` in a terminal one time store it in `.env` as `AP` Ignition's `ai.config` *password generator we long and random.)*
`database_url` password segment	`change_me_in_production`	The real Postgres password you set when provisioning the DB. *"Provisioning" = setting up.*	Whoever stood up Po Database Administrat `docker-compose.y` `POSTGRES_PASSWOR` `compose.yml`. Store `DATABASE_URL=pos` `chatbot:<REAL_PW` `ignition_chatbot` *the IP address or comp is running.* `postgres` *Postgres, with the asyn*
`service_env`	`"development"` *(the word "development" in quotes)*	`"production"` once you cut over from testing to real use	Hand-edit `.env` and *over" = stop testing, st*
`local_llm_endpoint`	`""` *(empty string)*	E.g. `http://vllm-host:8000/v1` if you're self-hosting an AI. *"Endpoint" = the web address.*	The host running you B12). **Leave empty if**

3.2 Secrets / `.env` (see `INSTALL.md` Part 2.1)

".env" = a small text file (named exactly `.env`, starting with a dot) in the project's main folder. It holds passwords and API keys that shouldn't be committed to source control.

Variable	Placeholder in INSTALL.md	What it actually is
`OPENAI_API_KEY`	`sk-PASTE_YOUR_OPENAI_KEY`	The password to your OpenAI account. Get one from https://platform.openai.com/api-keys — create a key scoped only to the org/project running this service. *("Scoped" = limited in what it can access, so if it leaks the damage is limited.)*

`AZURE_OPENAI_*` (4 vars)	commented out *(lines start with # so they're ignored)*	Only set if you use Azure OpenAI instead of public OpenAI. *(Azure = Microsoft's cloud version of OpenAI, often required by enterprise IT for data-residency reasons.)* Values come from your Azure OpenAI resource's "Keys and Endpoint" blade in the Azure portal.
`EMBEDDING_MODEL`	default fine	Leave alone unless you know why you'd change it. *"Embedding model" = the specific AI model used to convert text into the number lists from A2. Changing it means re-indexing every document.*

3.3 Ignition gateway — `ignition/perspective/gateway_wiring.py`

"Ignition gateway" = the server software from Inductive Automation that runs Perspective screens and talks to PLCs. "Perspective" = Ignition's web-based HMI (Human-Machine Interface = the screens operators look at). This file wires the chat screen to the chatbot.

Line	Placeholder	What to fix
45–50	`line_id = self.session.custom.activeLineId or "coater1"`	If you have more than one line, populate the **session custom prop** `activeLineId` from your view's binding to the active line tag. *("Session custom prop" = a per-user variable in Perspective. "Binding" = an Ignition feature that auto-updates one value from another. "Tag" = a single sensor reading or piece of state in Ignition.)* If only Coater 1 exists, leave as is.
165–175	`line_id = tag.tagPath.split("/")[1]`	Replace with however your **UDT path** encodes line. *("UDT" = User-Defined Type = a reusable folder structure of tags in Ignition. "Tag path" = the slash-separated address of a tag.)* E.g. if your tag path is `[default]Shaw/F0004/Coating/Coater1/Alarms/...`, then `split("/")[3]` gives `"Coater1"`. *(`split("/")` = "chop the string at every slash and give me a list." `[3]` = "the 4th piece" — counting starts at 0.)* Test once in the Ignition script console.

3.4 Ignition gateway — `ignition/scripts/config.py` (created from `INSTALL.md` Part 5)

Variable	Placeholder	Source

AI_SERVICE_URL	http://<HOST_IP_OF_AI_SERVER>:8000	The static IP / DNS of tl box running the FastAP container. *("Static IP" = an IP address that won't change. "DNS" = a name like* `chatbot.plant.loca` *that resolves to that IP. "FastAPI" = the Python web framework this servi is built on. "Container" = a Docker package.)* Get i from `ipconfig` on Windows or whatever your network team assigned.
API_KEY	PASTE_THE_SAME_API_KEY_FROM_2.1_ENV	The **same** value you pu in `.env API_KEY=...` above. *They must match exactly or Ignition will ge rejected.*
IGNITION_BASE_URL	http://<YOUR_GATEWAY_IP>:8088	Your Ignition Gateway's URL — the one you log into for Designer. *("Designer" = the desktop app you use to build Ignition projects. Port 8088 is Ignition's default.*
LINE_ID	"coater1"	Leave as `coater1` for the pilot; expand later. *("Pilot" = first limited rollout.)*
TAG_PROVIDER	"[UnifiedNamespace]"	From Designer → Tag Browser → top-level provider name (in brackets). *("Tag provider = the source of tags — could be a PLC connectio MQTT, etc. "UnifiedNamespace" = a common naming convention.)*
COATER1_ROOT	TAG_PROVIDER + "Shaw/F0004/Coating/Coater1"	Walk Tag Browser to th Coater1 UDT root and copy the path. *(The + here is string concatenati — gluing two pieces of te together.)*

3.5 Seed data — `service/scripts/seed_initial_data.py` (lines 50–

> "Seed data" = the starter content loaded into a fresh database.

The seeded line memory uses example equipment IDs (`coater1_zone3`, `coater1`) and product styles (`Style-A`, `Style-B`).

- If your real equipment IDs match these → no change needed.
- If they differ → search-and-replace before running `python -m scripts.seed_initial_data`. *(That command = "run the Python script located at `scripts/seed_initial_data.py` as a module." The `-m` flag is just how Python is told "run it the proper way.")*
- The seeded process facts (zone counts, fpm ranges, calibration drift notes) are **plausible defaults**. *("fpm" = feet per minute, the line speed unit.)* Have a process engineer review the ~12 entries and edit the strings before seeding. *("Strings" = pieces of text in code, written between quote marks.)*

3.6 Reference data — `scripts/seed_reference_data.sql`

> ".sql" = a file of SQL commands. SQL = Structured Query Language = the language databases speak.

`failure_modes` table is pre-populated with codes like `delam_hotpull`, `sag`, `coating_weight_var`, etc. These are the codes the LLM will use in classification + RCA. If your plant uses different vocabulary (e.g. `MAR` vs `delam_hotpull`), edit the SQL file **before** running it, because changing them later requires a **migration** of every historical `failure_mode_classifications.fm_code` value. *("Migration" = a careful script that updates existing data to match a new schema. They're annoying to write and risky to run on production data, so it's much easier to get the names right the first time.)*

---

4. What was asked for but is intentionally NOT built

- **Writing back to PLCs** — explicitly out of scope. Architecture principle #1 is "read-only." *("PLC" = Programmable Logic Controller = the industrial computer that actually runs the line. "Read-only" = the chatbot can look but can't touch.)*
- **Multi-line / multi-plant** — schema supports it (every table has a `line_id`), but seed data + Perspective view ship configured for Coater 1 only. Adding lines is a config exercise, not a code one.
- **Mobile / offline mode** — Perspective's responsive layout works on tablets; native mobile app and offline cache are not implemented. *("Responsive layout" = the screen resizes itself for phones/tablets. "Offline cache" = local storage so it still works when the network is down.)*
- **Voice input** — not implemented.

---

5. Test coverage snapshot

- **148 tests passing, 2 skipped, 0 failing** as of this commit. *("Tests" = automated checks that exercise the code. "Skipped" = intentionally not run, usually because they need an external service. "Commit" = a saved snapshot of the code in version control / git.)*

- Coverage is **unit + integration** with the DB layer mocked or in-memory. *("Unit test" = checks one small function. "Integration test" = checks several pieces working together. "Mocked" = replaced with a fake stand-in. "In-memory" = using a temporary database that lives only in RAM.)*
- **Not yet exercised:** real Postgres + pgvector roundtrip, real OpenAI call, real Ignition gateway pairing, load testing. *("Roundtrip" = save then read back. "Load testing" = hammering it with many requests at once to see if it holds up.)*

---

6. Recommended go-live sequence

1. **Replace every placeholder in §3.**
2. `docker compose up postgres` → run `setup_database.sql` then `seed_reference_data.sql` then `python -m scripts.seed_initial_data`. *("docker compose up postgres" = "Docker, please start the Postgres service defined in docker-compose.yml." Docker = a tool that runs programs in standardized boxes called containers.)*
3. `docker compose up service` → hit `GET /api/health` → expect 200 with `{"db": "ok", "embeddings": "ok"}`. *("GET" = the HTTP verb for "fetch this URL." "200" = HTTP status code for "success." The curly braces hold the JSON response — JSON is just a way of writing key/value data.)*
4. From a Python REPL on the service host: `curl -X POST /api/chat` with a hand-built `CuratedContextPackage` and the API key — confirm a grounded response with citations. *("REPL" = Read-Eval-Print Loop = an interactive Python prompt. "curl" = a command-line tool for making HTTP requests. "POST" = the HTTP verb for "send this data." "CuratedContextPackage" = the Python data structure holding the question + context. "Grounded" = backed up by citations rather than made up.)*
5. Wire Ignition (`INSTALL.md` Part 5), open the Perspective ChatView, send "what is the current state of zone 3?" — expect a real answer.
6. Run for one shift in **shadow mode** (operators see answers but a second engineer reviews each one via `/api/feedback`). *("Shadow mode" = the new system runs alongside the old way, not replacing it, so you can compare without risk.)*
7. Flip `outcome_closure_enabled=True` (already default), let one full 24-hour cycle pass, then check `v_rca_precision_daily` for your first precision number. *("Precision" = "of the times the chatbot said cause X, how often was X actually right?")*
8. Decide on B2 reranker, B5 HyDE, B6 self-consistency, B11 active learning, B13 eval harness based on what the first week of real traffic shows.

---

7. Mini glossary (the words that come up everywhere)

Term	Plain-English meaning
LLM	Large Language Model. The AI brain (e.g. GPT-4, Claude).
RAG	Retrieval-Augmented Generation. The pattern this whole system uses: search relevant documents, then ask the LLM to write an answer using only those documents.
Embedding	A list of ~1500 numbers that represents the *meaning* of a piece of text. Two pieces of text with similar meaning will have similar number lists.
Vector search	Searching by embedding similarity (meaning) rather than by exact words.
BM25	A classic keyword search algorithm. Like Ctrl-F but with relevance ranking.

Chunk	A bite-size piece of a longer document (e.g. one paragraph), stored in the DB so it can be retrieved on its own.
Citation	A pointer back to which chunk an answer came from, so a human can verify it.
Prompt	The written instructions given to the LLM ("You are an expert process engineer. Answer using only the context below...").
Tool / Tool call	A specific function the LLM is allowed to invoke mid-answer to get a real number or do a real lookup.
Token	The unit OpenAI charges by. Roughly ¾ of a word. 1000 tokens ≈ 750 English words.
Stub	A placeholder function or file that exists but doesn't do anything useful yet.
Schema	The database blueprint — which tables exist and what columns they have.
Migration	A script that upgrades an existing database to a new schema without losing data.
API	Application Programming Interface. The set of URLs/commands one program uses to talk to another.
Endpoint	One specific URL of an API (e.g. `/api/chat`).
Container / Docker	A standardized box that holds a program plus everything it needs to run. Docker is the tool that runs the boxes.
PLC	Programmable Logic Controller. The industrial computer that actually controls the machine.
HMI	Human-Machine Interface. The operator's screen.
Tag	A single named value in Ignition (e.g. "zone 3 temperature").
UDT	User-Defined Type. A reusable folder structure of tags.
Perspective	Ignition's web-based HMI module (the new style; runs in a browser).
Designer	Ignition's desktop app for *building* projects (as opposed to running them).
Failure mode (FM)	A standardized name for a type of defect (`delam_hotpull`, `sag`, etc.).
RCA	Root Cause Analysis. Figuring out *why* a defect happened, not just that it did.
Baseline	"What normal looks like." Used as the reference point for spotting deviations.